PRACTICAL 7-A

Object Oriented Programming

AIM:

To implement and understand the concepts specified in Practical 7 of the CEUE203 Object Oriented Programming practical list, using Java. The work covers Part A: Annotation Form Validator, Mini Test Runner, Column Mapper and Part B: Custom annotations and reflection validator.

CODE SCREENSHOT:

ColumnMapper.java

Part A > Practical 7-A > ColumnMapper.java > ...

```java
import java.lang.annotation.*;
import java.lang.reflect.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Column {
    String name();
}

class Person {
    @Column(name = "name")
    String name;
    @Column(name = "age")
    int age;

    public String toString() {
        return name + " (" + age + ")";
    }
}

public class ColumnMapper {
    Run | Debug
    public static void main(String[] args) throws Exception {
        String[] header = { "age", "name" }, row = { "20", "Anshul" };
        Person p = new Person();
        for (Field f : Person.class.getDeclaredFields()) {
            Column c = f.getAnnotation(Column.class);
            for (int i = 0; i < header.length; i++)
                if (c.name().equals(header[i])) {
                    f.setAccessible(true);
                    if (f.getType() == int.class)
                        f.setInt(p, Integer.parseInt(row[i]));
                    else
                        f.set(p, row[i]);
                }
        }
        System.out.println(p);
    }
}

```

Code Screenshot 1

FormValidator.java

Part A > Practical 7-A > FormValidator.java > ...

```java
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface NotBlank {
}

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface MaxLength {
    int value();
}

class SignupForm {
    @NotBlank
    String name = " ";
    @MaxLength(10)
    String username = "verylongusername";

    SignupForm(String n, String u) {
        name = n;
        username = u;
    }
}

public class FormValidator {
    Run | Debug
    public static void main(String[] args) throws Exception {
        SignupForm form = new SignupForm(n: " ", u: "verylongusername");
        List<String> errors = new ArrayList<>();
        for (Field f : form.getClass().getDeclaredFields()) {
            f.setAccessible(true);
            Object v = f.get(form);
            if (f.isAnnotationPresent(NotBlank.class) && (!(v instanceof String) || ((String) v).isBlank()))
                errors.add(f.getName() + " must not be blank");
            MaxLength max = f.getAnnotation(MaxLength.class);
            if (max != null && v instanceof String s && s.length() > max.value())
                errors.add(f.getName() + " exceeds " + max.value());
        }
        System.out.println(errors);
    }
}

```

Code Screenshot 2

MiniTestRunner.java

Part A > Practical 7-A > MiniTestRunner.java > ...

```java
import java.lang.annotation.*;
import java.lang.reflect.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Run {
}

class DemoTests {
    @Run
    void testOne() {
        System.out.println("testOne passed");
    }

    @Run
    void testTwo() {
        System.out.println("testTwo passed");
    }

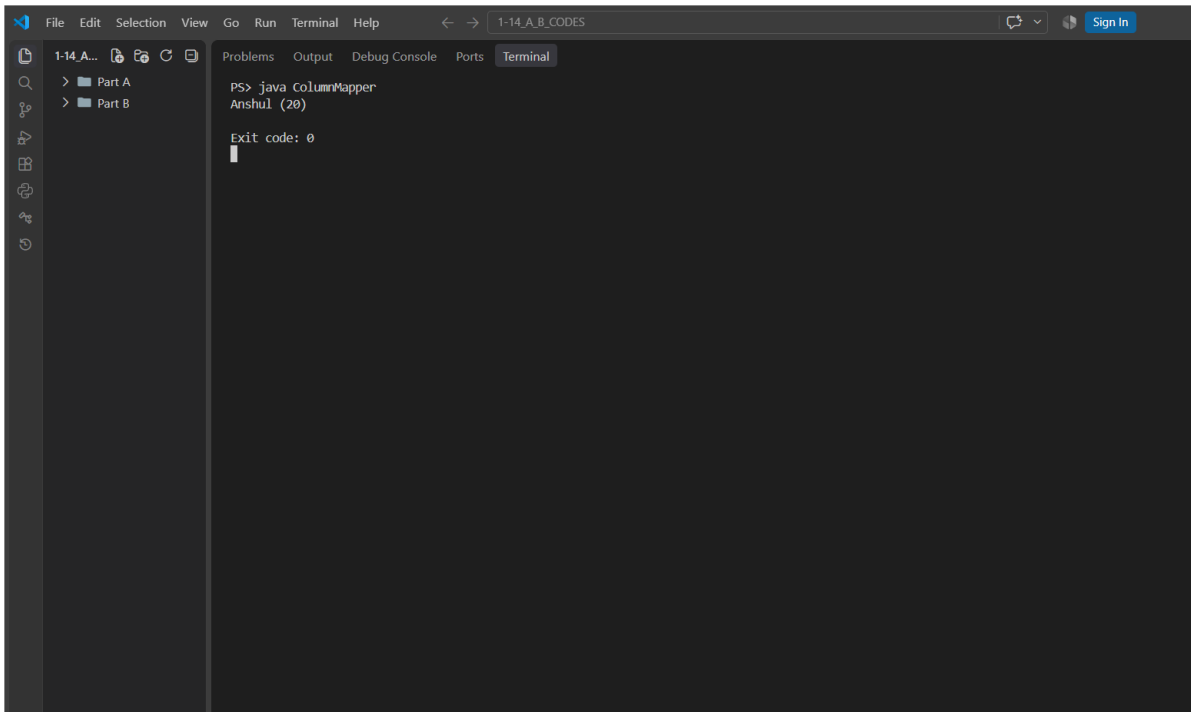
    void helper() {
        System.out.println("ignored");
    }
}

public class MiniTestRunner {
    Run | Debug
    public static void main(String[] args) throws Exception {
        Object test = new DemoTests();
        int count = 0;
        for (Method m : test.getClass().getDeclaredMethods())
            if (m.isAnnotationPresent(Run.class)) {
                m.invoke(test);
                count++;
            }
        System.out.println("Tests run: " + count);
    }
}

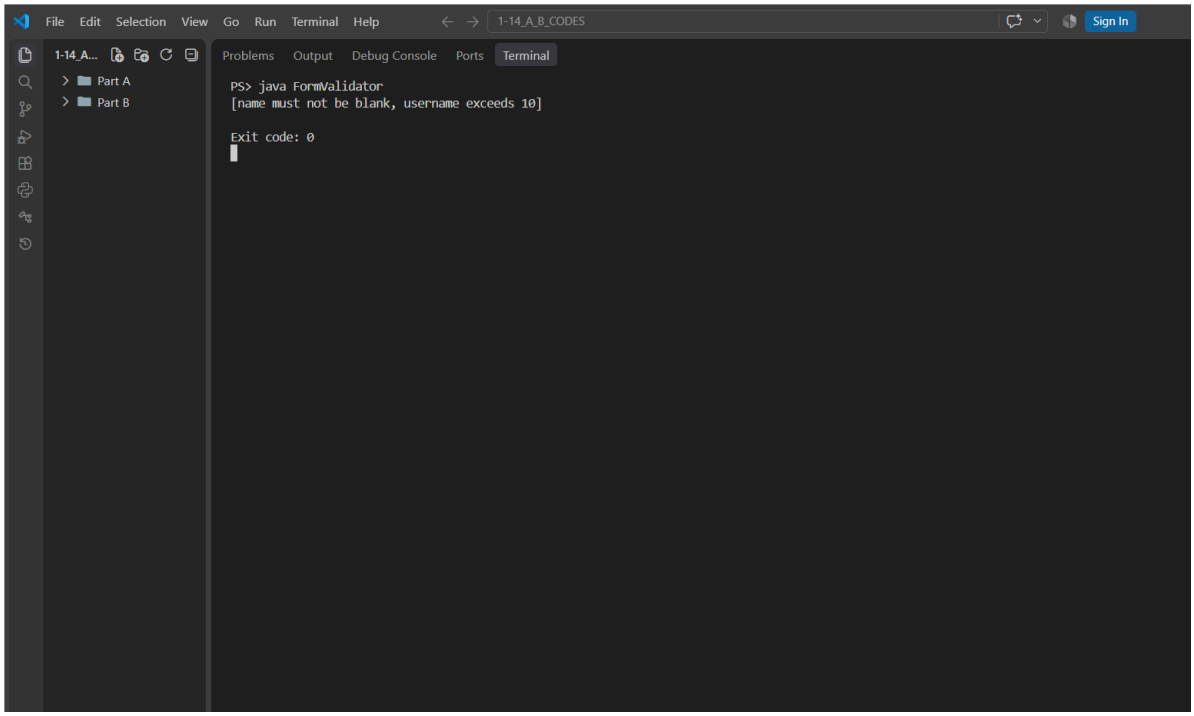
```

Code Screenshot 3

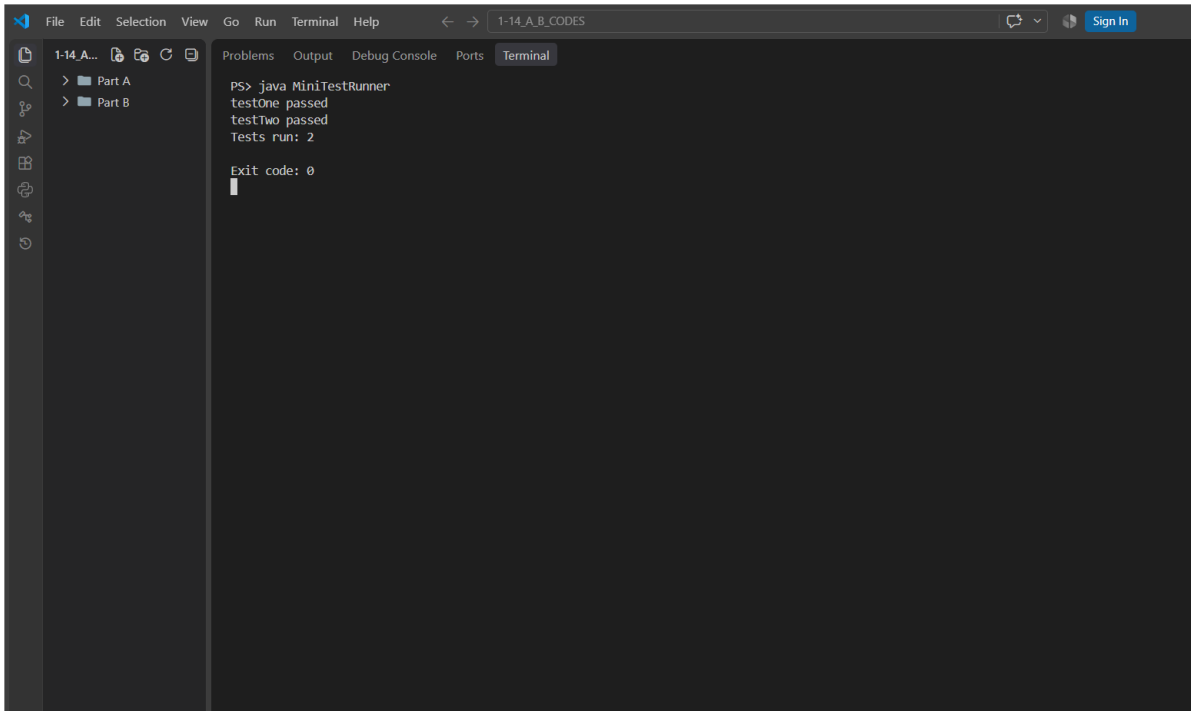
OUTPUT SCREENSHOT:



Output Screenshot 1



Output Screenshot 2



Output Screenshot 3

LEARNING OUTCOMES:

Create runtime annotations and inspect object fields through reflection for validation.